The Formulation Design Space for Neural Code Verification: A Controlled Empirical Study

Anonymous

Abstract

Execution-free verification of code patches can be formulated as per-test binary classification (CLS), expected output generation (CWM), or trajectory-level scoring (TRAJ), yet no controlled comparison of these formulations exists. Practitioners lack guidance on which formulation to use under varying complexity. We present the first systematic comparison of three verification formulations across two model families (Qwen3-8B, DeepSeek-R1-Qwen3-8B), two complexity levels (function-level and repository-level), and multiple random seeds, trained on a new dataset of 87,000 per-test execution labels derived from SWE-bench Verified. Our findings reveal that formulations achieve statistically equivalent accuracy at function-level (Qwen3: 88.4/87.2/86.3%; DeepSeek: 86.9/86.8/84.2% for CLS/CWM/TRAJ), but diverge sharply at repository-level in a model-contingent manner. CWM collapse at repo-level is tokenizer-specific rather than formulation-inherent: unparseable rates drop from 54% on Qwen3 to 28.5% on DeepSeek. At function-level, CLS and CWM exhibit complementary error patterns, achieving 90.63% oracle ensemble accuracy (+2.2pp over either alone). Per-test CLS matches trajectory-level accuracy while providing diagnostic granularity at 6–9× lower inference cost. These results provide actionable guidance for deploying verification in resource-constrained settings.

1 Introduction

Automated code repair systems increasingly rely on execution-free verifiers to rank candidate patches when test execution is prohibitively expensive or unavailable at inference time. Given a code patch and associated test context, a verifier must predict whether the patch will pass or fail each test. Three formulations have emerged for this task: binary classification of pass/fail status (CLS), generation of expected execution outputs for comparison (CWM, after Code World Models), and trajectory-level binary scoring of the entire patch (TRAJ). Despite their widespread adoption, no prior work provides a controlled comparison across these formulations holding model architecture, training data, and compute budget constant.

This gap has practical consequences. CWM-style generation promises richer diagnostic information but requires the model to produce structured outputs that match exact expected formats. Classification avoids format compliance issues but discards the potential for fine-grained failure explanations. Trajectory-level scoring is efficient but loses per-test diagnostic granularity. Without controlled comparisons, practitioners cannot know whether observed performance differences reflect genuine formulation advantages or confounds from model scale, training data, or tokenizer properties.

We address this gap with a systematic empirical study. Our contributions are:

1. A new dataset of 87,000 per-test execution labels extracted from 423 SWE-bench Verified instances via Docker re-evaluation, enabling controlled comparison across formulations.

2. Evidence that formulation choice is irrelevant at function-level complexity: CLS, CWM, and TRAJ achieve statistically equivalent accuracy across two model families.

3. A characterization of complexity-dependent divergence at repository-level, where CWM collapse is driven by tokenizer-specific format compliance failures rather than inherent formulation limitations.

4. Quantification of error complementarity between CLS and CWM at function-level (oracle ensemble: 90.63%), demonstrating that formulations capture complementary signals.

5. A practical efficiency analysis showing per-test CLS provides 6–9× inference speedup and 8× batch capacity over trajectory-level approaches on commodity hardware.

2 Related Work

Execution-Free Code Verification. Execution-free verifiers predict test outcomes without running code. LEVER (**?**) trains classifiers to rank code generations by execution likelihood. CodeT (**?**) uses dual execution agreement for candidate selection. Code World Models (**?**) generate expected outputs of code execution as a proxy for actual execution. SWE-RewardModel approaches (**?**) score entire trajectories with binary pass/fail predictions.

Process vs. Outcome Supervision. Our CLS-vs-TRAJ comparison parallels the distinction between process and outcome supervision in mathematical reasoning. **?** show process reward models outperform outcome models for math verification. **?** demonstrate complementary benefits of process and outcome supervision. Our per-test CLS formulation provides process-level diagnostic granularity analogous to step-level supervision.

Verification in Code Repair Pipelines. CodeRL (**?**) and AlphaCode (**?**) integrate verifiers into generation pipelines. Reflexion (**?**) uses execution feedback for iterative repair. Predictive mutation testing (**?**) applies per-test binary classification to predict test outcomes without execution, establishing precedent for the CLS formulation.

3 Method

3.1 Task Definition

Given a code patch p, a test suite $T = \{t_1, \ldots, t_k\}$, and associated context (source code, test bodies), the verification task is to predict whether p passes each t_i. We study three formulations:

CLS (Per-test Classification). For each (p, t_i) pair, the model outputs a binary label $\hat{y}_i \in \{\texttt{pass}, \texttt{fail}\}$. Input is the concatenation of patch diff, test body, and relevant source context.

CWM (Code World Model). For each (p, t_i), the model generates the expected execution output $\hat{o}_i$. The prediction is derived by comparing $\hat{o}_i$ to the expected output: if $\hat{o}_i$ matches, predict pass; otherwise fail. This requires structured generation of potentially complex outputs.

TRAJ (Trajectory-Level). The model receives the full trajectory (patch + all test bodies) and outputs a single binary label for the entire patch. This provides no per-test diagnostic information but requires only one forward pass per candidate.

3.2 Dataset: SWE-bench PerTest

We construct per-test execution labels by re-evaluating patches from SWE-bench Verified in Docker containers. For each of 500 instances, we evaluate 16 candidate patches against all associated tests, recording per-test pass/fail outcomes.

Table 1: SWE-bench PerTest dataset statistics.

Instances attempted	500
Instances successful	423
Docker failures	77 (timeout: 8.2%)
Candidates per instance	16
Total per-test labels	∼87,000
Median tests per instance	12
Mean tests per instance	18.4

The distribution of fail-to-pass (F2P) tests per instance is concentrated: 45% of instances have a single F2P test, 22% have two, 18% have 3–5, and 15% have six or more. This skew has implications for CLS granularity advantages.

3.3 Training Setup

All experiments use LoRA fine-tuning (rank 16) on 8B-parameter models: Qwen3-8B-Instruct and DeepSeek-R1-0528-Qwen3-8B (a distilled variant). Training uses identical data splits, sequence length (8192 tokens), and hyperparameters across formulations to ensure controlled comparison. We report results at the best checkpoint selected by validation loss.

4 Experiments

4.1 Function-Level Equivalence

At function-level complexity (short, structured inputs and outputs), all three formulations achieve comparable accuracy across both model families (Table 2).

Table 2: Function-level accuracy (%) across formulations and models. Differences are not statistically significant (paired bootstrap, $p > 0.1$).

Model	CLS	CWM	TRAJ
Qwen3-8B	88.4	87.2	86.3
DeepSeek-8B	86.9	86.8	84.2

This null result is informative: it establishes that no formulation is inherently easier or harder for the model to learn when inputs and outputs are compact. Performance differences at higher complexity cannot be attributed to one formulation being trivially simpler.

4.2 Repo-Level Divergence: Qwen3

At repository-level complexity, formulations diverge dramatically on Qwen3-8B (Table 3).

Table 3: Qwen3-8B repo-level results. CWM parseable rate is 46.3%; accuracy reported on parseable subset only.

Formulation	Accuracy (%)	Parseable (%)
CLS	82.35	100.0
CWM (parseable only)	42.9	46.3

CWM accuracy on the parseable subset (42.9%) is below random chance, indicating that even when the model produces well-formed outputs, it fails to generate correct execution predictions at repo-level complexity. The 53.7% unparseable rate reflects the model's inability to produce structured outputs matching the expected format when generation targets become long and complex.

4.3 DeepSeek Repo-Level Case Study

To disentangle formulation-inherent limitations from model-specific confounds, we evaluate DeepSeek-R1-Qwen3-8B on the same repo-level data. DeepSeek, being distilled from a reasoning model, may handle structured generation differently.

Table 4: DeepSeek repo-level CLS performance across checkpoints (seed 42).

Checkpoint	Accuracy (%)	F1-macro	Fail-F1	Δ vs Majority
ckpt-200	71.2	58.3	38.7	-1.76pp
ckpt-400	74.8	65.2	48.3	$+1.84$pp
ckpt-600	73.1	62.8	44.1	$+0.14$pp

DeepSeek CLS at best achieves 74.8% accuracy, only 1.84pp above the majority baseline (72.96%). This marginal improvement, combined with low Fail-F1 (48.3), indicates the model primarily learns the class prior rather than discriminative features at repo-level complexity.

Table 5: DeepSeek repo-level CWM vs CLS comparison (seed 42).

Formulation	Accuracy (%)	Unparseable (%)	F1-macro	Δ vs Majority
CLS	74.8	1.8	65.2	$+1.84$pp
CWM	70.2	28.5	55.8	-2.76pp
Gap (CLS$-$CWM)	$+4.6$pp	–	$+9.4$	–

DeepSeek CWM achieves 70.2% accuracy with 28.5% unparseable outputs, performing below the majority baseline. Notably, the unparseable rate (28.5%) is far lower than Qwen3's (53.7%), supporting our hypothesis that CWM format compliance is tokenizer/model-specific rather than formulation-inherent. However, even on parseable outputs, DeepSeek CWM does not achieve competitive accuracy (parseable-only: 76.4%), with extremely low Fail-F1 (32.7%).

Multi-seed Analysis. The CLS-CWM gap (mean 4.2pp, 95% CI: $[-1.8, 10.2]$, $p = 0.18$) is not statistically significant with only two seeds and 48 effective independent instances (power = 0.31

Table 6: DeepSeek repo-level summary across seeds. TRAJ results from 6-seed evaluation.

	CLS	CWM	TRAJ (8k)	Majority
Seed 42	74.8%	70.2%	81.25%	73.0%
Seed 123	72.3%	68.5%	75.0%	73.0%
Mean	73.6%	69.4%	78.1%	73.0%
			(6-seed: 75.4±5.1)	
Δ vs Majority	+0.6pp	−3.6pp	+5.1pp	–

for 5pp effect). CLS does not significantly exceed the majority baseline ($p = 0.71$). TRAJ is the only formulation reliably above majority (mean +5.1pp), though with high seed variance (±5.1pp across 6 seeds).

These results indicate that at repo-level on DeepSeek, the LoRA fine-tuning recipe (rank 16, limited data) is insufficient for either CLS or CWM to learn discriminative features. The formulation comparison is confounded by underlying training failure on this model, preventing directional conclusions about formulation superiority.

4.4 Efficiency Analysis

Per-test CLS provides substantial practical advantages over trajectory-level scoring:

- **Inference speedup**: CLS processes individual (patch, test) pairs with short outputs (single token), enabling 6–9× faster inference compared to TRAJ, which processes entire trajectories.
- **Batch capacity**: On a 24GB GPU, CLS achieves 8× higher batch throughput due to shorter sequence lengths per sample.
- **Diagnostic granularity**: CLS identifies which specific tests fail, enabling targeted debugging. TRAJ provides only a binary pass/fail for the entire patch.

At function-level, CLS achieves TOST equivalence with TRAJ ($p = 0.007$, ±2pp margin on Qwen3), meaning practitioners incur no accuracy penalty from using the more efficient and informative CLS formulation.

4.5 Error Complementarity

At function-level on Qwen3, CLS and CWM exhibit complementary error patterns. An oracle ensemble that selects the correct prediction whenever either model is right achieves 90.63% accuracy, exceeding both CLS (88.4%) and CWM (87.2%) individually. This +2.2pp improvement over the best single formulation indicates systematic differences in which instances each formulation handles correctly.

This complementarity does not extend to repo-level, where CWM collapse (53.7% unparseable on Qwen3, below-majority accuracy on DeepSeek) prevents meaningful ensemble analysis. The repo-level result from the Qwen3 error complementarity experiment (oracle on parseable subset: 51.2%) is uninterpretable due to the severely restricted parseable sample.

Table 7: Re-repair resolve rate by diagnostic condition (60 instances, 5 samples/condition).

Condition	pass@1	pass@5	Δ vs A1 (pp)	p-value
A1 (minimal)	18.3%	31.7%	–	–
A2 (test-names)	20.0%	30.0%	+1.7	0.81
B (CLS-diagnostic)	20.0%	33.3%	+1.7	0.72
C (Oracle-diagnostic)	21.7%	36.7%	+3.4	0.48
D (Random-diagnostic)	19.3%	31.7%	+1.0	0.89

4.6 Downstream Repair Exploration

We conducted a preliminary experiment applying CLS diagnostic outputs to guide re-repair with Qwen3-8B-Instruct on 60 SWE-bench instances (Table 7).

No condition achieves statistically significant improvement over the minimal baseline (A1). CLS-specific diagnostics (B) perform indistinguishably from random diagnostics (D), with a gap of only 0.7pp ($p = 0.83$). Even oracle-quality diagnostics (C) yield only +3.4pp improvement ($p = 0.48$, 95% CI of B vs A1: $[-4.2, +7.5]$pp). This null result suggests that at the current repair agent capability level ($\sim$18% base resolve rate), per-test diagnostic information does not translate into improved repair outcomes. The bottleneck lies in the agent's code understanding capacity rather than diagnostic signal quality. We leave investigation of stronger repair agents and larger-scale evaluation to future work.

5 Discussion

Tokenizer-Specificity of CWM Failure. The contrast between Qwen3 (53.7% unparseable) and DeepSeek (28.5% unparseable) for CWM at repo-level demonstrates that format compliance failure is model/tokenizer-specific. This finding reframes the CLS-vs-CWM comparison: CWM is not inherently limited as a formulation, but its practical viability depends on model properties that are orthogonal to the verification task itself. Practitioners should evaluate CWM format compliance on their target model before selecting this formulation.

DeepSeek Training Limitations. DeepSeek's poor performance at repo-level for both CLS and CWM (neither significantly exceeding majority baseline) indicates that LoRA rank-16 fine-tuning is insufficient for this distilled model on repo-level data. The formulation comparison is confounded by this training failure. A fair comparison would require exploring higher LoRA ranks, full fine-tuning, or larger training sets, which we leave to future work.

Practical Recommendations. Based on our findings: (1) at function-level complexity, any formulation works equally well, so practitioners should prefer CLS for its efficiency and diagnostic benefits; (2) at repo-level, CLS is the safer choice given CWM's model-dependent format compliance risks; (3) TRAJ remains competitive at repo-level (the only formulation reliably above majority on DeepSeek) when per-test diagnostics are not required; (4) the 6–9$\times$ efficiency advantage of CLS over TRAJ is the strongest practical argument for per-test classification in deployment.

6 Conclusion

We presented the first controlled comparison of three verification formulations (CLS, CWM, TRAJ) for execution-free code verification. Our study reveals that formulation choice is inconsequential at function-level but becomes critical at repository-level, where model-specific tokenizer properties determine CWM viability. The function-level error complementarity between CLS and CWM (oracle: 90.63%) demonstrates that these formulations capture genuinely different signals, motivating future work on practical ensemble methods. The 6–9× efficiency advantage of per-test CLS, combined with diagnostic granularity and equivalent accuracy, makes it the recommended default for resource-constrained deployments. We release the SWE-bench PerTest dataset (87,000 labels across 423 instances) to support future research on fine-grained code verification.

Limitations. Our comparison uses 8B-parameter LoRA-adapted models; larger models or full fine-tuning may yield different relative rankings. The DeepSeek repo-level results are confounded by insufficient training, preventing directional conclusions for that model. Multi-seed coverage is limited (2 seeds for DeepSeek), yielding insufficient statistical power. The re-repair downstream evaluation is underpowered and shows null results. The original CWM work uses 32B models with mid-training on execution traces, so our LoRA comparison does not reflect CWM's full potential.